

I. Plantilla básica

01. C++ Básico

```
#include <bits/stdc++.h>

using namespace std;

void solve() {
    // Aquí va tu código
}

int main() {
    ios_base::sync_with_stdio(0); // Optimización
    cin.tie(0); // Optimización

    int numCasos;
    cin >> numCasos;
    while (numCasos--) {
        solve();
    }

    return 0;
}
```

Para problemas de casos múltiples (100% de los problemas de blitzcoding)

II. Tipos de datos (Básicos)

```
int main() {
    int entero = 10; // Desde -10^9 hasta 10^9 (1.000.000.000)
    long long enteroGrande = 100000; // Desde -10^18 hasta 10^18
    double decimal = 3.1416;
    bool esPrimo = true;
    string saludo = "Hola mundo";
    char letra = 'a';

    return 0;
}
```

III. Operaciones matemáticas

```
int main() {
    int suma = 20 + 10;
    int resta = 20 - 10;
    int mult = 20 * 8;
    int div = 80 / 4;

    suma += 10; // Agregarle 10 a la suma acumulada
    resta -= 5; // Quitarle 5 a la resta
    // También se puede acumular con: *= /= %=

    int modulo = 10 % 3; // residuo de dividir 10 entre 3 es 1

    int potencia = pow(2, 3); // 2 al cubo es 8
    int raizCuadrada = sqrt(9); // raiz de 9 es 3

    return 0;
}
```

IV. Condicionales y Operadores Lógicos

```
int main() {
    int edad = 18;
    string nombre = "Axel";

    if (edad >= 18 && nombre == "Axel") {
        cout << "Se cumplen las dos condiciones";
    } else if (edad >= 18 || nombre == "Axel") {
        cout << "Solo se cumple una condición";
    } else {
        cout << "No se cumple ninguna condición";
    }

    return 0;
}
```

V. Bucles

```
int main() {
    for (int i = 0; i < 10; i++) {
        // Se repite 10 veces
    }

    for (int i = 0; i < 10; i += 2) {
        // Se repite 5 veces, el i va de 2 en 2
    }

    int j = 10;
    while (j--) {
        // Se repite 10 veces
    }

    int k = 0;
    while (k < 10) {
        // se repite 10 veces
        k++;
    }

    return 0;
}
```

I. Vectores

vector<TIPO_DATO> nombre

```
int main() {
    // Declaración y Constructor
    vector<int> numeros;           // {}
    vector<int> numeros(4);       // {0, 0, 0, 0}
    vector<int> numeros(4, 10);   // {10, 10, 10, 10}
    vector<int> numeros = {1, 2, 3, 4};

    // Acceso
    //      0  1  2  3
    cout << numeros[2];          // {1, 2, [3], 4} Imprime 3
    cout << numeros.front();     // {[1], 2, 3, 4} Imprime 1
    cout << numeros.back();      // {1, 2, 3, [4]} Imprime 1

    // Modificadores
    numeros.push_back(5); // {1, 2, 3, 4, 5} Agrega un nuevo elemento al final
    numeros.pop_back();   // {1, 2, 3, 4} Elimina el elemento del final
    numeros.clear();      // {} Elimina todos los elementos

    // Capacidad
    cout << numeros.size(); // Cant. de elementos del vector = 4
    numeros.resize(8); // Cambia el tamaño a 8 = {1, 2, 3, 4, 0, 0, 0, 0}
    cout << numeros.empty(); // Da true cuando el vector está vacío o false si no.

    // Iterar por vectores
    for (int i = 0; i < numeros.size(); i++) {
        cout << numeros[i] << " ";
    }

    for (int& elemento : numeros) {
        cout << elemento << " ";
    }

    return 0;
}
```

II. Queue (Cola)

Una queue o cola es una estructura de datos que sigue el principio FIFO (First In, First Out), donde el primer elemento que entra es el primero en salir.

```
int main() {
    // Declaración
    queue<int> cola;           // {}

    // Modificadores
    cola.push(1);              // {1}
    cola.push(2);              // {1, 2}
    cola.pop();                // {2} Elimina el primer elemento

    // Acceso
    cout << cola.front();      // Primer elemento
    cout << cola.back();       // Último elemento

    // Capacidad
    cout << cola.size();       // Cantidad de elementos
    cout << cola.empty();      // true si está vacía

    return 0;
}
```

III. Stack (Pila)

Una stack o pila es una estructura de datos que sigue el principio LIFO (Last In, First Out), donde el último elemento que entra es el primero en salir.

```
int main() {
    // Declaración
    stack<int> pila;           // {}

    // Modificadores
    pila.push(1);             // {1}
    pila.push(2);             // {2, 1}
    pila.pop();               // {1} Elimina el último elemento

    // Acceso
    cout << pila.top();       // Elemento superior de la pila

    // Capacidad
    cout << pila.size();      // Cantidad de elementos
    cout << pila.empty();     // true si está vacía

    return 0;
}
```

IV. Set (Conjunto)

Un set es una estructura de datos que mantiene elementos únicos y ordenados.

```
int main() {
    // Declaración
    set<int> numeros;          // {}
    set<int> numeros = {4, 2, 1, 3, 4}; // {1, 2, 3, 4} Se ordena automático sin duplicados

    // Modificadores
    numeros.insert(5);        // {1, 2, 3, 4, 5}
    numeros.erase(3);         // {1, 2, 4, 5}
    numeros.clear();           // {}

    // Búsqueda
    auto it = numeros.find(2); // Retorna iterador a la posición del 2
    int cant = numeros.count(2); // Retorna 1 si existe el elemento 2, 0 si no

    // Capacidad
    cout << numeros.size();    // Cantidad de elementos
    cout << numeros.empty();   // true si está vacío

    return 0;
}
```

V. Map (Diccionario)

Un map es una estructura de datos que asocia **claves únicas** con **valores**, similar a un diccionario. Se ordenan automáticamente y se evitan duplicados en las **claves**.

map<TipoDato_CLAVE, TipoDato_VALOR> nombre;

```
int main() {
    // Declaración
    map<string, int> edades; // {}
    map<string, int> edades = { {"Juan", 25}, {"Ana", 30} };

    // Modificadores
    edades["Maria"] = 28; // Agrega o actualiza valor
    edades.erase("Juan"); // Elimina un elemento
    edades.clear(); // {}

    // Búsqueda
    auto it = edades.find("Ana"); // Retorna un iterador que apunta al elemento con la clave "Ana"
    int edad = edades["Ana"]; // Accede al valor de la edad de Ana

    // Capacidad
    cout << edades.size(); // Cantidad de pares
    cout << edades.empty(); // true si está vacío

    return 0;
}
```

VI. Pair (Parejas)

Un pair es una estructura simple que permite almacenar dos valores de cualquier tipo. Es útil cuando necesitas mantener dos valores relacionados juntos.

```
int main() {
    // Declaración
    pair<int, string> par; // (0, "")
    pair<int, string> par = {1, "uno"}; // (1, "uno")

    // Acceso
    cout << par.first; // Imprime 1
    cout << par.second; // Imprime "uno"

    return 0;
}
```

I. Iteradores

Tema único de C++. Relacionado con punteros, acceso a memoria y referencias.

- `begin()` Devuelve un iterador al **primer elemento** del contenedor.
- `end()` Devuelve un iterador al **elemento que sigue al último**.
- `rbegin()` Devuelve un iterador al **último elemento**.
- `rend()` Devuelve un iterador al **elemento que precede al primero**.

```
int main() {
    vector<int> numeros = {1, 2, 3, 4, 5, 6};

    // Iteradores
    auto Inicio = numeros.begin();    // {[1], 2, 3, 4, 5, 6}
    auto antesFinal = numeros.end();  // {1, 2, 3, 4, 5, 6} []

    // Iteradores Inversos
    auto Final = numeros.rbegin();    // {1, 2, 3, 4, 5, [6]}
    auto antesInicio = numeros.rend(); // [] {1, 2, 3, 4, 5, 6}

    // Operaciones con Iteradores
    auto Inicio = numeros.begin();    // {[1], 2, 3, 4, 5, 6}
    cout << *Inicio; // Imprime el valor 1
    Inicio++; // Mueve el iterador a la derecha {1, [2], 3, 4, 5, 6}
    Inicio--; // Mueve el iterador a la izquier. {[1], 2, 3, 4, 5, 6}
    Inicio + 3; // Mueve el iterador 3 a la derecha {1, 2, 3, [4], 5, 6}
    Inicio - 2; // Mueve el iterador 2 a la izquier. {1, [2], 3, 4, 5, 6}

    return 0;
}
```

II. Ordenamiento (Básico)

```
int main() {
    vector<int> numeros = {5, 3, 6, 4, 2, 1};

    sort(numeros.begin(), numeros.end()); // {1, 2, 3, 4, 5, 6} Forma ascendente
    sort(numeros.rbegin(), numeros.rend()); // {6, 5, 4, 3, 2, 1} Forma descendente

    return 0;
}
```

III. Ordenamiento (Forma personalizada)

El tercer parámetro se conoce como "función lambda" y devuelve **true** si el primer elemento debe ir antes que el segundo.

```
int main() {
    vector<string> palabras = {"perro", "gato", "elefante", "zorro", "abeja", "raton"};

    sort(palabras.begin(), palabras.end(), [&](string a, string b) {
        // Si tienen la misma longitud, ordenar alfabéticamente
        if (a.size() == b.size()) {
            return a < b;
        }

        // Ordenar primero por longitud de la palabra
        return a.size() < b.size();
    });

    // {"gato", "abeja", "perro", "raton", "zorro", "elefante"}

    return 0;
}
```

IV. Búsqueda Binaria (Básica)

```
int main() {
    vector<int> numeros = {1, 2, 3, 4, 5, 6, 7, 9, 10};
    int x = 7; // Número que buscamos

    // Obligatorio ordenar los elementos antes de hacer la búsqueda binaria
    sort(numeros.begin(), numeros.end());

    // Izquierda      Derecha
    int left = 0, right = numeros.size() - 1;

    while (left <= right) {
        // Medio
        int mid = left + (right - left) / 2;

        if (numeros[mid] == x) {
            cout << "La posición del número es: " << mid;
            return 0;
        }

        if (numeros[mid] < x) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    cout << "No se encontró el número";

    return 0;
}
```

Necesitamos tres punteros: izquierdo, derecho y medio. En cada iteración, comparamos el elemento buscado con el elemento medio y descartamos la mitad del rango donde sabemos que no puede estar el elemento. Este proceso se repite hasta encontrar el elemento o determinar que no existe.

V. Búsqueda Binaria (Funciones de C++)

```
int main() {
    vector<int> numeros = {1, 2, 3, 4, 5, 6, 7, 9, 10};

    // Obligatorio ordenar los elementos antes de hacer la búsqueda binaria
    sort(numeros.begin(), numeros.end());

    // Función de C++
    bool existe = binary_search(numeros.begin(), numeros.end(), 5); // true
    bool existe = binary_search(numeros.begin(), numeros.end(), 8); // false

    // Iterador a la primera ocurrencia de 5 - {1, 2, 3, 4, [5], 6, 7, 9, 10};
    auto it = lower_bound(numeros.begin(), numeros.end(), 5);
    cout << *it << endl; // 5

    // Iterador a la posición después de 5 - {1, 2, 3, 4, 5, [6], 7, 9, 10};
    auto it = upper_bound(numeros.begin(), numeros.end(), 5);
    cout << *it << endl; // 6

    // Si no lo encuentran, ambas funciones devuelven el número más cercano posterior
    // {1, 2, 3, 4, 5, 6, 7, [9], 10};
    auto it = lower_bound(numeros.begin(), numeros.end(), 8);
    auto it = upper_bound(numeros.begin(), numeros.end(), 8);
    cout << *it << endl; // 9 (porque no hay 8 en el contenedor, va al siguiente)

    return 0;
}
```

La búsqueda binaria es un algoritmo eficiente para **encontrar un elemento en un contenedor ordenado**, dividiendo repetidamente el espacio de búsqueda a la mitad.

Este algoritmo tiene una complejidad de $O(\log n)$, lo que lo hace mucho más rápido que la búsqueda lineal para conjuntos grandes de datos.

I. Divisores de un Número N

```
int main() {
    long long N = 100000000000; // (long long si N es mayor a 10^9)

    vector<long long> divisores;
    for (long long div = 1; div * div <= N; div++) {
        if (N % div == 0) {
            divisores.push_back(div);

            if (N / div != div) {
                divisores.push_back(N / div);
            }
        }
    }

    return 0;
}
```

II. Primalidad

```
int main() {
    long long N = 13; // (long long si N es mayor a 10^9)

    for (long long div = 2; div * div <= N; div++) {
        if (N % div == 0) {
            cout << "No es primo";
            return 0;
        }
    }
    cout << "Sí es primo";

    return 0;
}
```

IV. Máximo Común Divisor

En inglés Greatest Common Divisor

```
int main() {
    cout << gcd(30, 48) << endl;

    return 0;
}
```

Desde C++ 20 en adelante

V. Mínimo Común Múltiplo

En inglés Least Common Multiple

```
int main() {
    cout << lcm(4, 6) << endl;

    return 0;
}
```

Desde C++ 20 en adelante

⚠ Blitzcoding no reconoce estas funciones, debes crearlas tú mismo.
En la siguiente página está la manera de hacerlo.

III. Factorización

```
int main() {
    long long N = 18; // (long long si N es mayor a 10^9)

    // pair<factor primo, exponente>
    vector<pair<long long, int>> factores;
    if (N == 1) {
        cout << "1 1";
        return 0;
    }

    for (long long p = 2; p * p <= N; p++) {
        if (N % p == 0) {
            int exp = 0;
            while (N % p == 0) {
                exp++;
                N /= p;
            }
            factores.push_back({ p, exp });
        }
    }
    if (N != 1) factores.push_back({ N, 1 });

    // Imprimir las parejas
    for (int i = 0; i < factores.size(); i++) {
        cout << factores[i].first << " " << factores[i].second << endl;
    }

    return 0;
}
```

VI. GCD y LCM para C++ 11.

```
#include <bits/stdc++.h>

using namespace std;

int gcd(int a, int b) {
    if (a > b) swap(a, b);
    if (a == 0) return b;
    return gcd(b % a, a);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    cout << gcd(18, 27) << endl;
    cout << lcm(4, 6) << endl;

    return 0;
}
```

Forma de crear tú mismo las funciones de gcd y lcm. Recomendable su uso si se desconoce la versión del compilador. En blitzcoding es obligatorio.

Si obtienes un error **Compilation Error** usando gcd() o lcm(), puede ser un indicador que el compilador es una versión antigua como C++ 11, C++ 14 o C++ 17. Tendrás que crear tú mismo las funciones.